תכנות מונחה עצמים ב-Java

תכנות מונחה עצמים (OOP - Object Oriented Programming) מארגן את הקוד סביב אובייקטים, שכל אחד מהם משלב נתונים (שדות) והתנהגות (מתודות). ל-OOP ארבעה עקרונות: כימוס, ירושה, פולימורפיזם והפשטה.

מחלקה ואובייקט

מחלקה (Class) היא תבנית שמגדירה אילו שדות ומתודות יהיו לאובייקט. **אובייקט (Object)** הוא מופע ממשי של המחלקה, שנוצר עם המילה new.

```
public class Car {
    String model;
    int year;

    public Car(String model, int year) {   // בנאי
        this.model = model;
        this.year = year;
    }

    public void drive() {
        System.out.println(model + " is driving");
    }
}

Car myCar = new Car("Toyota", 2022);
myCar.drive();
```

בנאי (Constructor) הוא מתודה מיוחדת ששמה זהה לשם המחלקה, ואין לה ערך החזרה. היא רצה פעם אחת כשיוצרים אובייקט. המילה this מתייחסת לאובייקט הנוכחי.

כימוס (Encapsulation)

כימוס פירושו הסתרת הנתונים הפנימיים של המחלקה, וגישה אליהם רק דרך מתודות מבוקרות. מגדירים את השדות כ-`private` ומוסיפים מתודות `getter` ו-`setter`.

```
public class BankAccount {
    private double balance;

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}
```

הרשאות גישה (Access Modifiers):

- `public` - נגיש מכל מקום.
- `private` - נגיש רק מתוך המחלקה עצמה.
- `protected` - נגיש מתוך המחלקה, מתת-מחלקות ומאותה חבילה (package).
- ללא הגדרה (default) - נגיש רק מתוך אותה חבילה.

ירושה (Inheritance)

ירושה מאפשרת למחלקה חדשה לקבל את השדות והמתודות של מחלקה קיימת, באמצעות המילה extends. המחלקה היורשת נקראת תת-מחלקה, והמחלקה שממנה יורשים נקראת מחלקת-על.

```
public class Animal {
    public void eat() {
        System.out.println("eating...");
    }
}

public class Dog extends Animal {
    public void bark() {
        System.out.println("woof");
    }
}
```

ב-Java מחלקה יכולה לרשת ממחלקה אחת בלבד (אין ירושה מרובה של מחלקות). המילה super פונה למחלקת-העל, למשל כדי לקרוא לבנאי שלה: super(name);.

פולימורפיזם (Polymorphism)

פולימורפיזם פירושו "צורות רבות": אותה קריאה למתודה מתנהגת אחרת לפי סוג האובייקט בפועל. יש שני סוגים:

- **Overriding (דריסה)** - תת-מחלקה כותבת מחדש מתודה של מחלקת-העל, עם אותה חתימה. מסמנים אותה ב-@Override. ההחלטה איזו מתודה תרוץ מתקבלת בזמן ריצה.
- **Overloading (העמסה)** - כמה מתודות באותה מחלקה עם אותו שם אבל פרמטרים שונים. ההחלטה מתקבלת בזמן קומפילציה.

```
class Animal { void sound() { System.out.println("..."); } }
class Cat extends Animal {
    @Override
    void sound() { System.out.println("meow"); }
}

Animal a = new Cat();
a.sound();   // meow
```

הפשטה: מחלקה מופשטת וממשק

מחלקה מופשטת (abstract class) היא מחלקה שאי אפשר ליצור ממנה אובייקט. היא יכולה להכיל מתודות מופשטות (בלי מימוש) לצד מתודות רגילות ושדות.

```
public abstract class Shape {
    abstract double area();
}

public class Circle extends Shape {
    double r;
    double area() { return Math.PI * r * r; }
}
```

ממשק (interface) מגדיר "חוזה": רשימת מתודות שמחלקה מתחייבת לממש, עם המילה `implements`. מחלקה יכולה לממש כמה ממשקים, וכך Java מאפשרת סוג של ירושה מרובה.

```
interface Flyable { void fly(); }
interface Swimmable { void swim(); }

class Duck implements Flyable, Swimmable {
    public void fly() { System.out.println("flying"); }
    public void swim() { System.out.println("swimming"); }
}
```

ההבדל בין abstract class ל-interface: מחלקה יורשת ממחלקה מופשטת אחת בלבד, אבל יכולה לממש ממשקים רבים. למחלקה מופשטת יכולים להיות שדות רגילים ובנאי, ולממשק לא. משתמשים במחלקה מופשטת כשיש קשר "הוא סוג של" וקוד משותף, ובממשק כשרוצים להגדיר יכולת.